

C Preprocessors

22 July 2026 20:32

The C program is known as 'Source Code'. When the source code is passed through 'preprocessor', it creates 'Expanded Source Code' as per the preprocessor directives used in the source code. Each preprocessor directive begins with a '#' hash symbol. Following preprocessor directives may be used in the source code:

- a. Macro expansion
- b. File inclusion
- c. Conditional compilation
- d. Miscellaneous Directives

Macro Expansion

```
#include<stdio.h>
#define PI 3.1428
int main(int argc, char const *argv[])
{
    float r = 6.25, area;
    area = PI * r * r;
    printf("Area of circle = %f\n", area);
    return 0;
}
```

In the statement:

```
#define PI 3.1428
```

PI is called 'macro template', whereas, 3.1428 is called 'Macro expansion'. During preprocessing every macro template gets replaced with its corresponding macro expansion. Usually macro templates are written in capital letters. This makes it easy for programmers to identify macro template when reading through the program.

And now an important question - why use **#define** at all? Suppose the constant 3.1428 appears many times in your program. Someday you may wish to change all these values with more accurate 3.142857. For this you have to go through the program and manually change each occurrence of the constant. However, if you have defined PI in a **#define** directive, you only need to change the hash defined directive to:

```
#define PI 3.1428
```

Once done, the change will be made in all occurrences of Pi during preprocessing. This convenience may not matter for small programs shown above, but with large programs, macro definitions are almost indispensable.

You may feel that same purpose could have been served. Had we used a variable Pi instead of macro template PI. But for three reasons, it would have been a bad idea.

Firstly, it is inefficient since the compiler can generate faster and more compact code for constant than it can for variables. Secondly, using a variable for what is really a constant encourages sloppy thinking - If something never changes, it is hard to imagine it as a variable. And thirdly, there is always a danger that the variable may inadvertently get altered somewhere in the program. So it is no longer a constant that you think it is.

```
#include<stdio.h>
#define PI 3.1428
#define AND &&
#define OR ||
#define NOT !
#define not_equal !=
#define True 1
#define False 0
```

← It helps us to write the code in a very much in readable form.

```
int main(int argc, char const *argv[])
{
    float r = 6.25, area;
    area = PI * r * r;
    printf("Area of circle = %f\n", area);
    return 0;
}
```

Macro with Arguments

The macros. That we have used so far are simple macros. Macros can have arguments, just as function can. Here is a program that illustrates this fact.

```
#include<stdio.h>
#define PI 3.1428
#define AND &&
#define OR ||
#define NOT !
#define not_equal !=
#define True 1
#define False 0
#define AREA_OF_CIRCLE(r) (PI * r * r)
int main(int argc, char const *argv[])
{
    float r = 6.25, area, r1 = 3.45;
    area = AREA_OF_CIRCLE(r);
    printf("Area of circle = %f\n", area);
    area = AREA_OF_CIRCLE(r1);
    printf("Area of circle = %f\n", area);
    return 0;
}
```

The preprocessor would replace every $\text{AREA_OF_CIRCLE}(x)$ with $(PI * x * x)$. As it does this, x would be substituted with the argument that we use in the macro. Thus $\text{area} = \text{AREA_OF_CIRCLE}(r1)$ would be replaced with $\text{area} = (PI * r1 * r1)$

```
}
```

Output:

Area of circle = 122.765625

Area of circle = 37.407177

```
#include<stdio.h>
#define PI 3.1428
#define AND &&
#define OR ||
#define NOT !
#define not_equal !=
#define True 1
#define False 0
#define AREA_OF_CIRCLE(r) (PI * r * r)
#define ISDIGIT(y) (y >= '0' AND y <= '9')
#define ISCAPITAL(ch) (ch >= 'A' AND ch <= 'Z')
#define ISSMALL(ch) (ch >= 'a' AND ch <= 'z')

int main(int argc, char const *argv[])
{
    float r = 6.25, area, r1 = 3.45;
    area = AREA_OF_CIRCLE(r);
    printf("Area of circle = %f\n", area);
    area = AREA_OF_CIRCLE(r1);
    printf("Area of circle = %f\n", area);
    char ch;
    printf("Enter any character: ");
    scanf("%c", &ch);
    if(ISDIGIT(ch)){
        printf("The given character is digit.");
    }
    else if(ISCAPITAL(ch)){
        printf("The given charcter is Capital letter.");
    }
    else if(ISSMALL(ch)){
        printf("The given charcter is small letter.");
    }
    else{
        printf("The given character is something else.");
    }
    return 0;
}
```

Output:

```

Area of circle = 122.765625
Area of circle = 37.407177
Enter any character: Q
The given charcter is Capital letter.
PS C:\Users\bluej\Documents\Students\Sou
Chakraborty\CProgramming\ForGit\Coding\
rams\PreprocessorDirectives> & "C:\Users
j\Documents\Students\Soumodip Chakraborty
ogramming\ForGit\Coding\CPrograms\Prepro
rDirectives\two.exe"
Area of circle = 122.765625
Area of circle = 37.407177
Enter any character: t
The given charcter is small letter.
PS C:\Users\bluej\Documents\Students\Sou
Chakraborty\CProgramming\ForGit\Coding\
rams\PreprocessorDirectives> & "C:\Users
j\Documents\Students\Soumodip Chakraborty
ogramming\ForGit\Coding\CPrograms\Prepro
rDirectives\two.exe"
Area of circle = 122.765625
Area of circle = 37.407177
Enter any character: 9
The given character is digit.
PS C:\Users\bluej\Documents\Students\Sou
Chakraborty\CProgramming\ForGit\Coding\
rams\PreprocessorDirectives> & "C:\Users
j\Documents\Students\Soumodip Chakraborty
ogramming\ForGit\Coding\CPrograms\Prepro
rDirectives\two.exe"
Area of circle = 122.765625
Area of circle = 37.407177
Enter any character: ^
The given character is something else.

```

Here are some important points to remember while writing macros with arguments.

- Do not give a space between macro template and its argument in **#define**.
- The entire macro expansion should be enclosed within parenthesis.
- Macros can be split into multiple lines with a ('\')back slash present at the end of each line except the last. For example:

```

#include<stdio.h>
#define PI 3.1428
#define AND &&
#define OR ||
#define NOT !
#define not_equal !=
#define True 1
#define False 0
#define AREA_OF_CIRCLE(r) (PI * r * r)
#define ISDIGIT(y) (y >= '0' AND y <= '9')
#define ISCAPITAL(ch) (ch >= 'A' AND ch <= 'Z')
#define ISSMALL(ch) (ch >= 'a' AND ch <= 'z')
#define HLINE(start, end) \
    do{\
        int i = (start);\
        for(; i < end; i++)\
            printf("%c", 196);\
    }while(0);

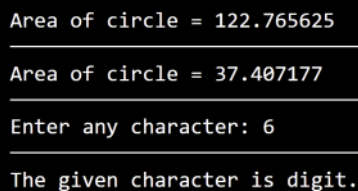
```

```

int main(int argc, char const *argv[])
{
    float r = 6.25, area, r1 = 3.45;
    area = AREA_OF_CIRCLE(r);
    printf("\nArea of circle = %f\n", area);
    HLINE(1, 30);
    area = AREA_OF_CIRCLE(r1);
    printf("\nArea of circle = %f\n", area);
    HLINE(1,30);
    char ch;
    printf("\nEnter any character: ");
    scanf("%c", &ch);
    HLINE(1,30);
    if(ISDIGIT(ch)){
        printf("\nThe given character is digit.\n");
    }
    else if(ISCAPITAL(ch)){
        printf("\nThe given charcter is Capital letter.\n");
    }
    else if(ISSMALL(ch)){
        printf("\nThe given charcter is small letter.\n");
    }
    else{
        printf("\nThe given character is something else.\n");
    }
    HLINE(1,30);
    return 0;
}

```

Output:



```

Area of circle = 122.765625
Area of circle = 37.407177
Enter any character: 6
The given character is digit.

```

- d. If you are unable to debug a macro, you should view the expanded code of the program to see how the macro is getting expanded. If your source code is present in file two.c, then the expanded source code would be stored in two.I . You need to generate this file at the command prompt by saying:

```
gcc -E two.c -o two.i
```

This command invoked the C preprocessor, which generates the expanded source code and stores it in the file called two.I, You can now open this file and see the expanded source code.

